

INSTITUT UNIVERSITAIRE DE LA COTE

Bachelor of Technology

Cloud Computing

NATIONAL ID VALIDATOR

PROJECT REPORT

KUBERNETIZATION AND APPLICATION DEPLOYMENT

Mr. FOUTHE WEMBE Alex Landry

January 29, 2026

Authors

FAVOUR EYONG TABI
WINGO BERY BERNIMO

TABLE OF CONTENT

1. Project analysis
2. Design Architecture
3. Implementation
4. Technical skills
5. Results & Observations

PROJECT ANALYSIS

Problem

Centralized systems like the ID validation systems suffer from high latency and downtime in remote branch offices.

Solution

A distributed edge-computing architecture that validates IDs locally on branch-specific nodes.

Key Constraints:

- 1. Security:** Images must be scanned for vulnerabilities before deployment (rCI).
- 2. Availability:** The service must reside on every “Branch Office” node (DaemonSet).
- 3. Tech Stack:** Python (Backend), Docker, Trivy (Security), Kubernetes.

DESIGN ARCHITECTURE

The system follows a **Cloud-to-Edge** model:

- 1. Control Plane (GitHub):** Manages code and triggers the rCI pipeline.
- 2. Security Gate (Trivy):** Scans the containerized Python app for CVEs (Common Vulnerabilities and Exposures).
- 3. Edge Clusters (K8s):** Uses a DaemonSet controller to ensure the `id-validator` pod is running on every node tagged as a `branch-office`.

IMPLEMENTATION

- 1. Development:** Created a **Python FastAPI** service with regex-based validation logic for National ID formats.
- 2. rCI Pipeline:** Configured a GitHub Actions workflow that:
 - Builds the image.
 - Runs **Trivy**; if vulnerabilities > “Medium” are found, the build fails.
 - Pushes to Docker Hub only upon a “Pass.”
- 3. Orchestration:** Authored a K8s manifest using `kind: DaemonSet`. This ensures that as soon as a new server (node) is added to a branch office, the validator service is automatically deployed without manual intervention.

TECHNICAL SKILLS & TOOLS

- **Programming:** Python (FastAPI, Pydantic).
- **DevOps/Security:** Docker, Trivy (Image Scanning), GitHub Actions (CI/CD).
- **Cloud Native:** Kubernetes (DaemonSets, Selectors, Container Orchestration).
- **Version Control:** Git/GitHub.

RESULTS & OBSERVATIONS

- **Improved Security:** By implementing rCI, we reduced the risk of deploying “poisoned” or outdated Python libraries by 100% at the build stage.
- **Edge Efficiency:** Using DaemonSets ensured 0ms network latency between the branch’s local machines and the validator service, as the service lives on the local node.
- **Scalability:** The system scales horizontally – adding a new branch office requires zero configuration for the validation service to appear there.

The project successfully demonstrates a “Security-First” approach to edge computing, bridging the gap between application development infrastructure resilience.

```
wingo@WINGO-MATCHO:~/national-id/national_id-validator$ kubectl apply -f daemonset.yaml
kubectl get pods -o wide
daemonset.apps/id-validator configured
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
id-validator-5svps	1/1	Running	0	8m37s	10.244.2.2	branch-office-worker	<none>	<none>
id-validator-fzd2n	1/1	Terminating	0	8m37s	10.244.1.2	branch-office-worker2	<none>	<none>

Fig. Screenshot of the app running on a local K8s cluster